WALCHAND COLLEGE OF ENGINEERING, SANGLI

(Government Aided Autonomous Institute)

WALCHAND LINUX USERS' GROUP

WLUG

Community | Knowledge | Share

# METAMORPHOSIS

LIMITED SEATS

SAILING THROUGH THE CONTAINERS ERA

EXCITING PRIZES

11 & 12 APRIL

- DOCKER DRIFT
- DOCKER DEEPDIVE

- GO KICKSTART
- GO UNLEASHED

REGISTER HERE

SPONSORED BY

FOSS UNITED

CONNECT WITH US

₹349 | ₹649 | ₹899

MAIN & MINI CCF

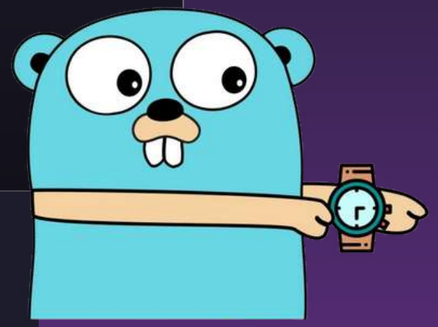
FOR ANY QUERY
9579047160
7219685991

| | | | | | |
|---|---|---|---|---|---|
| Ms. Pooja Bhosale<br>President<br>Walchand Linux Users' Group | Dr. A. J. Umbarkar<br>Staff Advisor<br>Walchand Linux Users' Group | Dr. R. R. Rathod<br>HoD Information Technology | Dr. A. R. Surve<br>Staff Advisor and HoD Computer Science and Engineering | Dr. M. M. Khot<br>Dean (SW) | Dr. S. S. Rathod<br>Director |

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# GO KICKSTART

It's Go Time



# CONTENTS


- Introduction to Golang
- History of Golang
- Why use Golang?
- Data Types in Golang
- Variables and Constants in Golang
- Control Flow Statements in Golang

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# INTRO TO GOLANG

# GOLANG

- Developed By Google
- Creators of GO :-

  Rob Pike

  Ken Thomson

  Robert Greismer
- Go was released as Open Source in 2009

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# WHY GO ?

## TECHNICAL PERFORMANCE

- Fast Execution
- Built-in Concurrency
- Memory Efficiency
- Static Typing & Safety

## DEVELOPER EXPERIENCE

- Minimalistic Syntax
- Fast Compilation
- Robust Standard Library
- Scalability at Core

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# WHO USES GOLANG ?

Meta

# INSTALLING GOLANG

```
sudo apt update
sudo apt install golang-go -y
```

# GO VERSION / ENV

- Shows the installed Go version on your system

```
go version
```

- Displays Go environment variables and configuration settings

```
go env
```

# FIRST PROGAM

**main.go**

```go
package main
import "fmt"

func main() {
    fmt.Println("Welcome to Metamorphosis 2K26 !!")
    fmt.Println("Sailing through Containers ERA !!")
}
```

Run the Program using

go run main.go

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# DATA TYPES IN GOLANG

## BASIC DATA-TYPES

- Integer
- String
- Float
- Boolean

## DERIVED DATA-TYPES

- Array
- Slices
- Map
- Structure

# VARIABLES

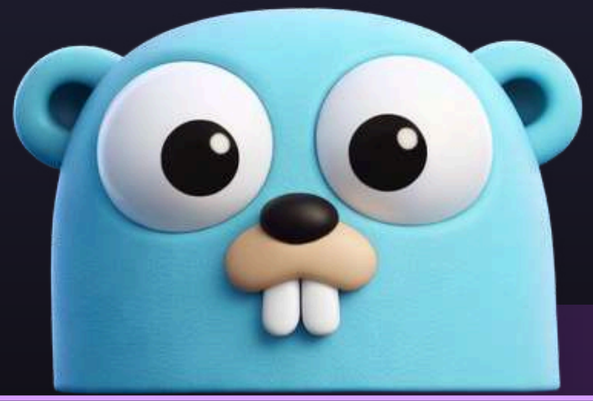
- A variable is a container to store the data
- Each variable has it’s own type and value
- Declared using “var” or “ := “

# WAYS OF DECLARING VARIABLES

```go
// Using var
var a int = 10

// Type Inference
var a = 10

// Short Declaration
a := 10

//Multiple Variables
var a, b, c int = 1, 2, 3
a, b := 1, 2
```

# CONSTANTS

- Constants in Golang are immutable
- A constant is a variable whose value cannot be changed after declaration
- Declared using const



# WAYS OF DECLARING CONSTANTS

```go
//Single Constants
const pi float64 = 3.14

//Multiple Constants
const a, b = 1, 2

//Grouped
const (
    x = 10
    y = 20
)
```

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# CONTROL FLOW STATEMENTS

- Conditional Statements
- Loops
- Control Statements

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# CONDITIONAL STATEMENTS

if Statement

else if Statement

switch Statement

# IF STATEMENT

- Executes code based on condition

```go
var num = 10

if num&1 == 0{
    fmt.Println("Number is Even!!")
}

```

# ELSE IF STATEMENT

- else if in Go is used to check multiple conditions sequentially when the previous if condition is false.

```go
var num = 0

if num == 0 {
    fmt.Println("0 is neither even nor odd!!")
}

else if num&1 == 0{
    fmt.Println("Number is Even!!")
}

else{
    fmt.Println("Number is Odd!!")
}
```

# SWITCH STATEMENT

- Clean multi-condition control structure with automatic breaks and flexible usage

```go
x := 10

switch {
case x&1 == 0:
    fmt.Println("Number is Even !!")
case x&1 != 0:
    fmt.Println("Number is Odd!!")
}
```

# LOOPS

- for loop : used to repeat a block of code multiple times

```go
i := 0
for i < 10 {
    i++
    fmt.Println(i)
}
```

# LOOPS

- while-like loop :- for loop used as while loop

```
i := 0
for i < 5 {
    i--
}
```

# GUESS ??

```
i := 0
for i < 5 {
    i--
}
```

# EXPLANATION

- Initially → i = 0
- Condition → i < 5  (true)
- loop runs:

  i-- → i = -1

  Check again → -1 < 5

  i-- → -2

  -2 < 5
- This keeps going forever i.e infinite loop

# BREAK

- Used to exit the loop at a particular moment

```
for i := 0; i < 10; i++ {
    if i == 5 {
        break
    }
}
```

# CONTINUE

- Used to skip the current iteration

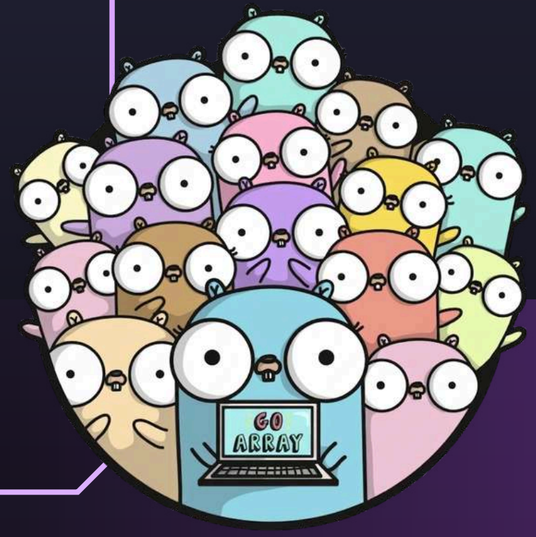
```go
for i := 0; i < 10; i++ {
    if i == 5 {
        continue
    }
}
```

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# ARRAYS

# ARRAYS

- Collection of elements of the same data type, stored in fixed size
- By default an array is zero-valued
- “len” returns the length of an array
- Appear in the form [v1 v2 v3 ...] when printed with fmt.Println()



# ARRAYS

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 7 | 32 | 36 | 50 | 54 | 72 | 81 | | | | | | | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

# ARRAYS

```
var array_name [array_size] data_type
OR
array_name := [array_size] data_type {elements}
```

# ARRAYS

```go
var numbers [5]int
        OR
arr := [3]int{1, 2, 3}
```

# SLICES

- Important data type in Go
- Lightweight data structure
- Variable-length sequence
- To create an empty slice with non-zero length, use the builtin make()

# SLICES

```
slice_name := make([]data_type, size)
                OR
slice_name := [] data_type {elements}
```

# SLICES

```go
nums := make([]int, 5)  // slice of size
        OR
nums := []int{10, 20, 30, 40}
```

# SLICES

- Appending a Slice : Returns a slice containing one or more new values

Copying a Slice

- Create an empty slice c of the same length as s
- Copy into c from s

# SLICES

```go
nums = append(nums, 5, 6)
OR
src := []int{10, 20, 30}
dest := make([]int, 3)
copy(dest, src)
```

# SLICES

Slice Operator

- “slice” operator with the syntax slice [low:high]
- For eg. arr[:7] means all the elements in an array upto index 6

# SLICES

```
slice_name [lower_index : upper index]

sub := nums[1:4]
```

# MAP

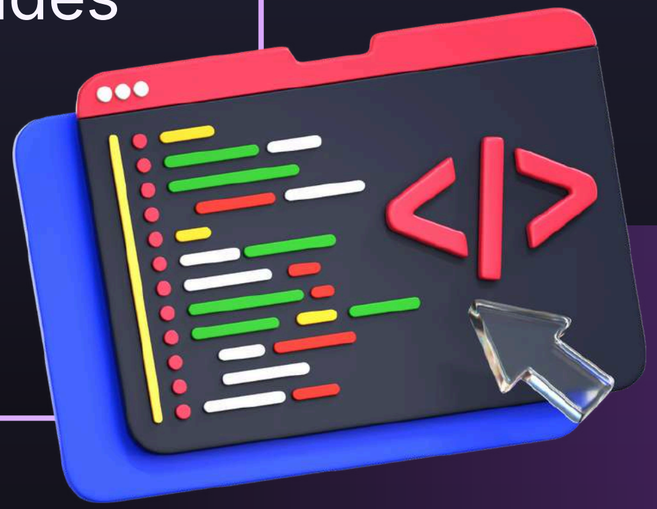
- Built-in associative data type ( key/value pairs)
- Syntax to create any empty map
  make(map[key-type]val-type)
- Setting the value to map - name[key]
- `len` returns the number of key/value pairs when called on a map

# MAP SYNTAX

```
map_name := make(map[key data_type]value_data_type)
```

# FUNCTIONS

- Functions are the block of code in a program that can perform a specific task
- It gives user the ability to reuse the same code
- Saves excessive use of memory, and provides readability



# FUNCTIONS

```
// Functions Declaration
func function-name(parameter-list) return-type{
      //function body
}

// Functions Calling
function-name(parameter-list)
```

# FUNCTIONS ARGUMENTS

- The parameters passed to function are called actual parameters, whereas the parameters received by a function are called formal parameters.

# POINTERS

- Go supports pointers
- By default an array is zero-valued
- & -> pointer to a variable that points its address
- * -> value of variable pointed by pointer

# FUNCTIONS

Call by value

- Value of actual parameters is copied into functions formal parameter
- Any changes made inside function do not impact the actual parameters

# FUNCTIONS

Call by reference

- Both the actual and formal parameters refer to the same location
- Changes made inside function are actually reflected in the actual parameters

# FUNCTIONS

Multiple return types

- A function in go can return more than one value of same or different data types

```
func function-name(parameters) (return-types){
//function body
}
```

# FUNCTIONS

Custom type declaration

- To define a custom type declaration, we use the type keyword
- Example: struct, []string, []bytes

# STRUCTS

- A struct is a user defined data type that consists of one or more variables
- By default an array is zero-valued
- Fields can have different data types

# STRUCTS

Defining structs:

```
type struct-name struct{
      field-name field-type
      field-name field-type
}
```

# STRUCTS

Initializing variables:

```
type Person struct {
Name string
Age int
}
item1 := Person{"Rahul"
,49}
item2 := Person{Age: 49,Name: "Rahul"}
item3 := Person{Name: "Rahul"}
item3.Age=49
item4 := Person{}
item4.Name=
"Rahul"
item4.Age=49
```

# ERROR HANDLING

- Errors are a part of any program
- An error tells if something unexpected happens
- Errors also help maintain code stability, maintainability
- Without errors, the programs we use today will be extremely buggy due to a lack of testing

# ERROR HANDLING

Creating Error

- GoLang errors package has a function called New() which can be used to create errors easily

Syntax:

```
func returnError() error {
    return errors.New("This is an Error!")
}
```

# ERROR HANDLING

Panic and Recover

- Panic occurs when an unexpected wrong thing happens. It stops the function execution
- Recover is the opposite of it
- It allows us to recover the execution from stopping

# ERROR HANDLING

Defer:

- Used to delay the execution of a function or a statement until the nearby function returns. In simple words
- Defer will move the execution of the statement to the very end inside a function

# ERROR HANDLING

```go
package main

import "fmt"

func main() {
    defer fmt.Println("World")
    fmt.Println("Hello")
}
/*
Output:
Hello
World
*/
```

# ERROR HANDLING

- No matter what happens later → file will always close

```go
file, err := os.Open("data.txt")
if err != nil {
    return
}
defer file.Close()
```

# GOROUTINES

- Used to handle concurrent Programmes
- Goroutines can be thought of as lightweight threads
- The cost of creating a Goroutine is tiny when compared to a thread. Hence Go applications mainly have thousands of Goroutines running concurrently

# GOROUTINES

Syntax:

- Put a keyword “go” before calling the name of Function

# GOROUTINES

WITHOUT goroutines:Runs sequentially

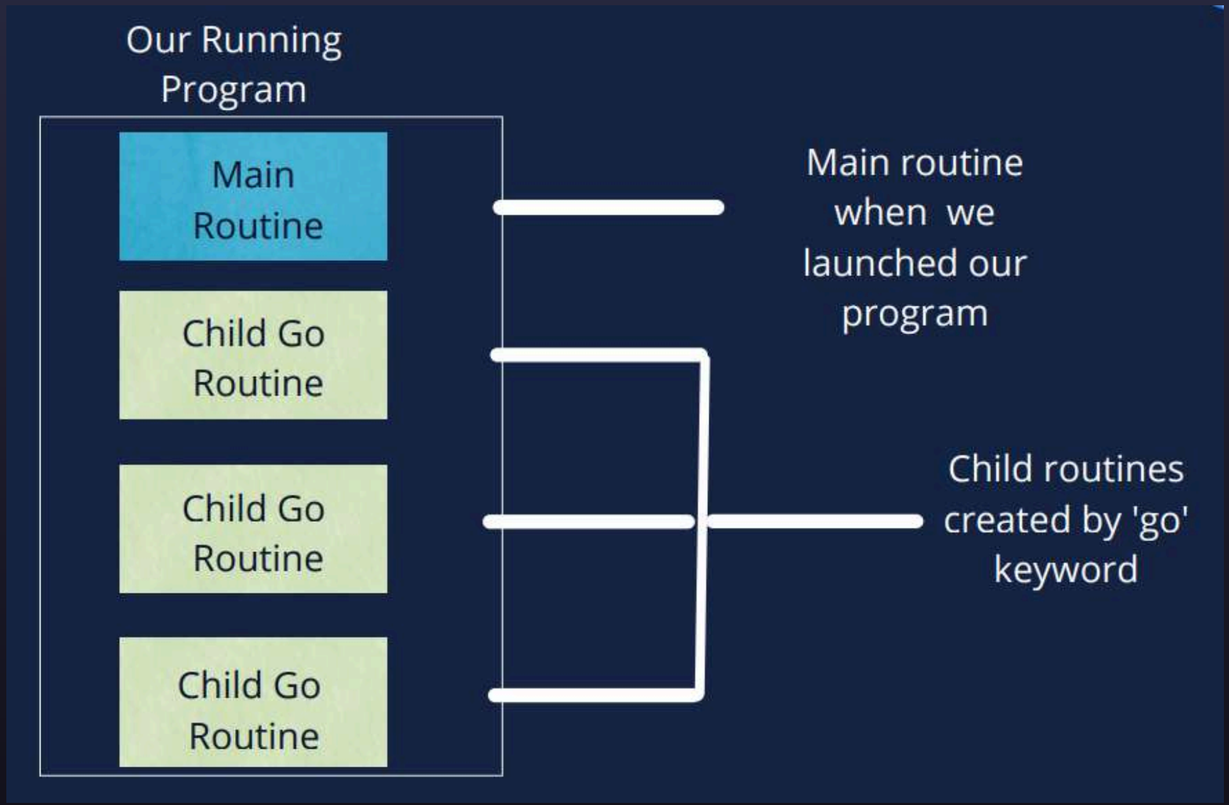
```go
func task1() {
    fmt.Println("Task 1")
}
func task2() {
    fmt.Println("Task 2")
}

func main() {
    task1()
    task2()
}
/*Output:
Task 1
Task 2
```

# GOROUTINES

WITH goroutines:Runs at the same time

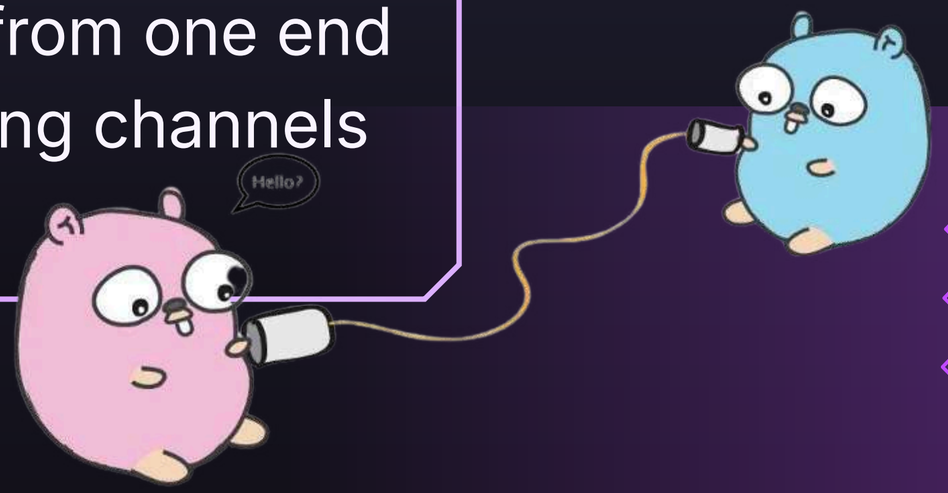
```go
func task1() {
    fmt.Println("Task 1")
}
func task2() {
    fmt.Println("Task 2")
}

func main() {
    task1()
    task2()
}
/*Output:
Task 1
Task 2
```

# GOROUTINES

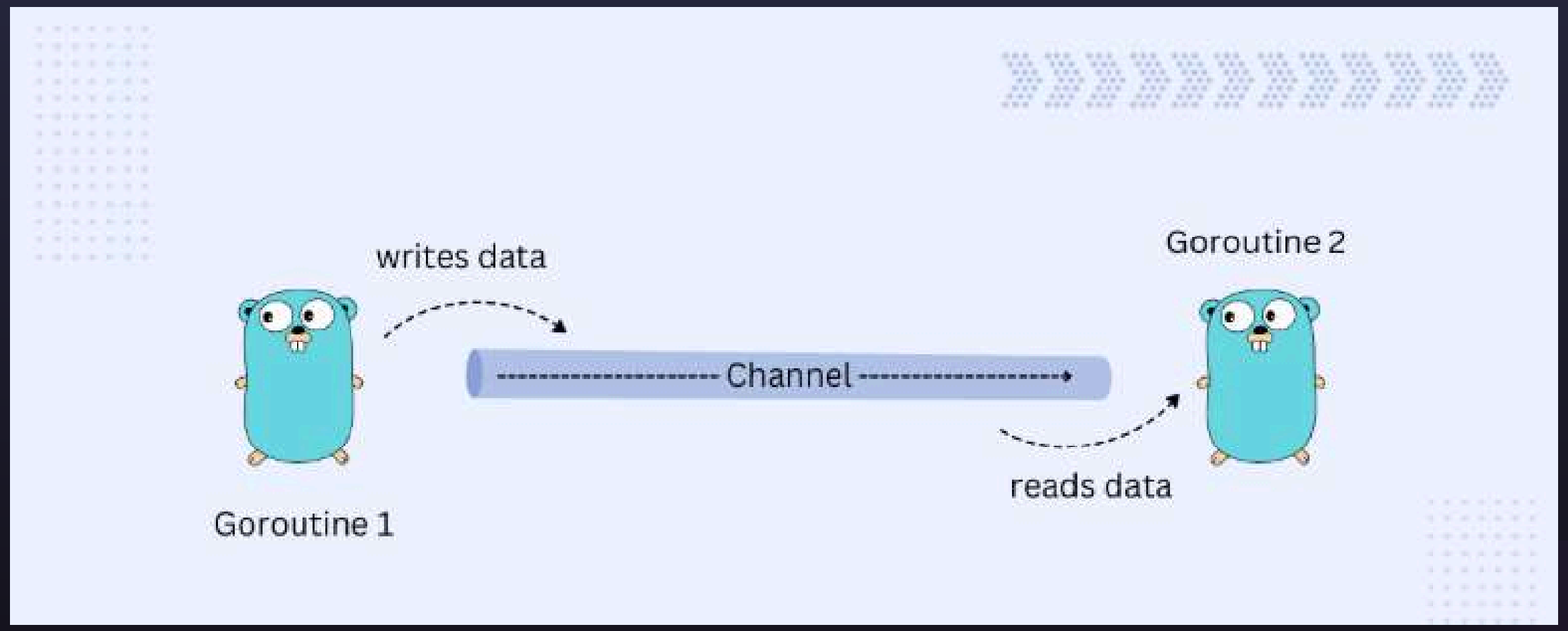

Our Running Program
Main Routine
Child Go Routine
Child Go Routine
Child Go Routine
Main routine when we launched our program
Child routines created by 'go' keyword

# CHANNELS

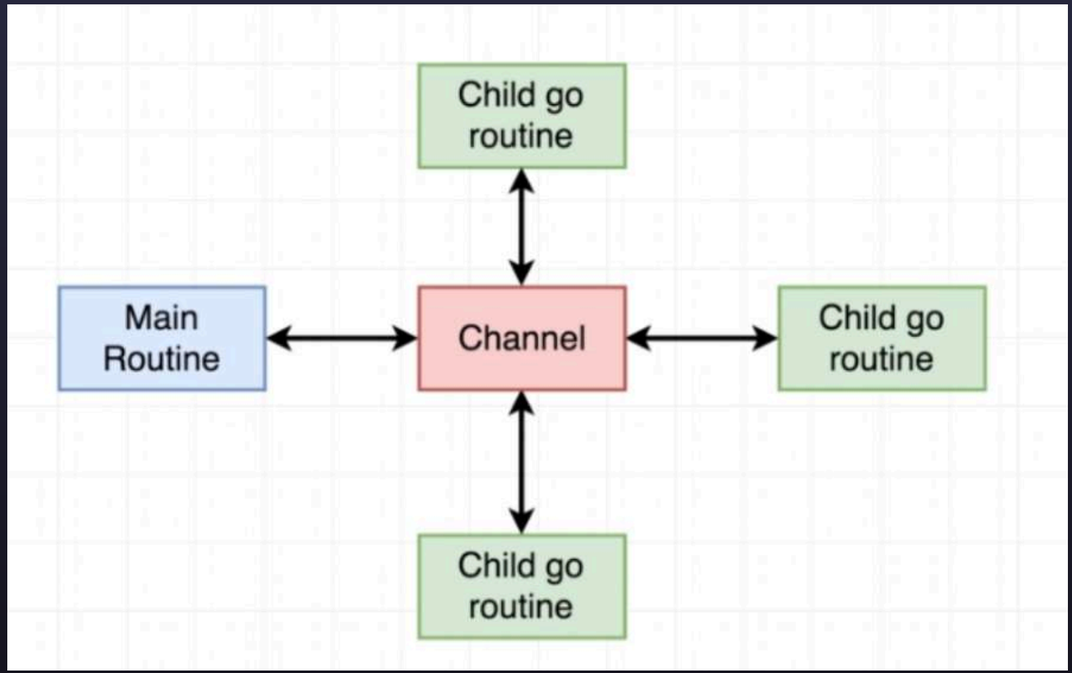
- Pipes using which Goroutines communicate
- A programming construct that allows us to move data between different goroutines
- Similar to how water flows from one end to another in a pipe, data can be sent from one end and received from the other end using channels



# CHANNELS



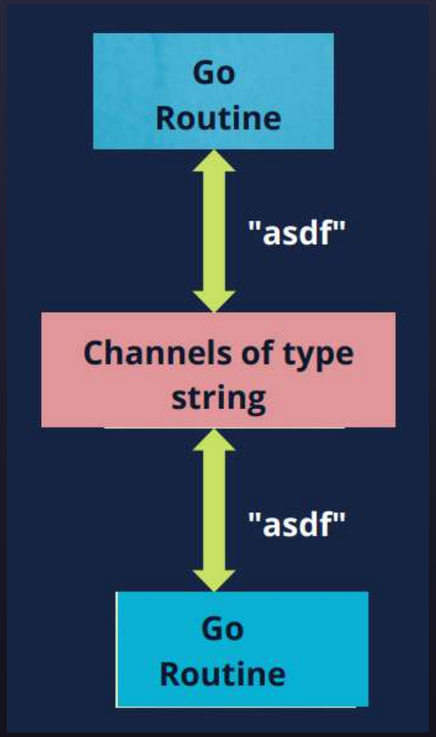
# CHANNELS

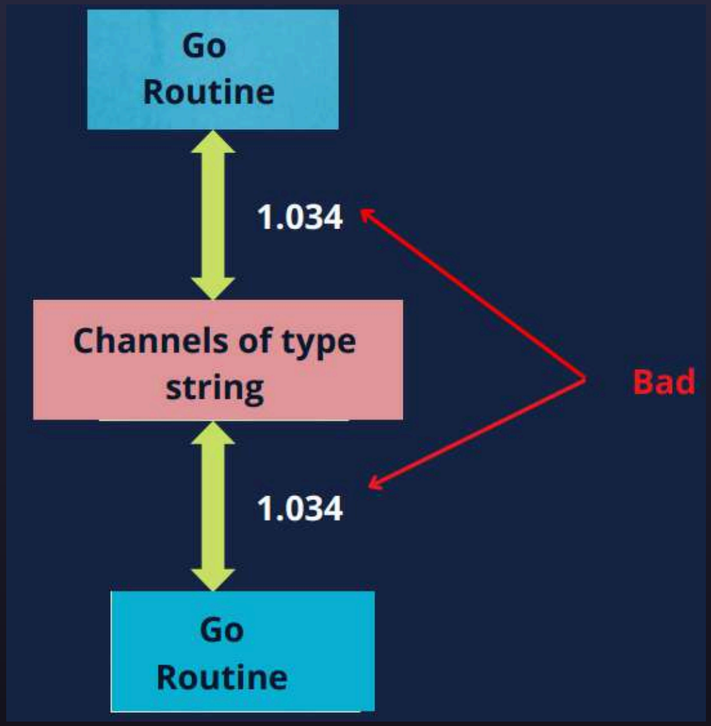

Child go routine
Main Routine
Channel
Child go routine
Child go routine

# CHANNELS

- Data can be sent from one end and received from the other end using channels
- Each channel has a type associated with it. This type is the type of data that the channel is allowed to transport

# CHANNELS


Go
Routine
"asdf"
Channels of type
string
"asdf"
Go
Routine


Go
Routine
1.034
Channels of type
string
Bad
1.034
Go
Routine

# CHANNELS

Sending, Receiving data

- data := <- a // read from channel a
  a <- data // write to channel a
- The direction of the arrow with respect to the channel specifies whether the data is sent or received

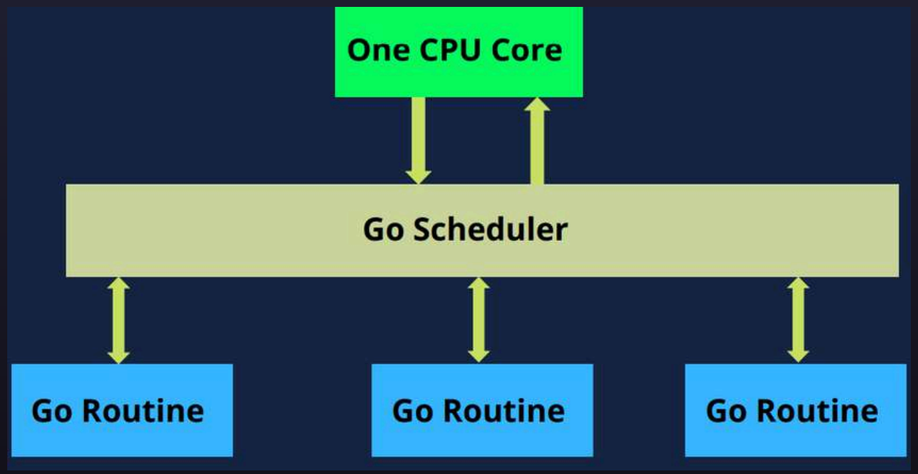
# CHANNELS

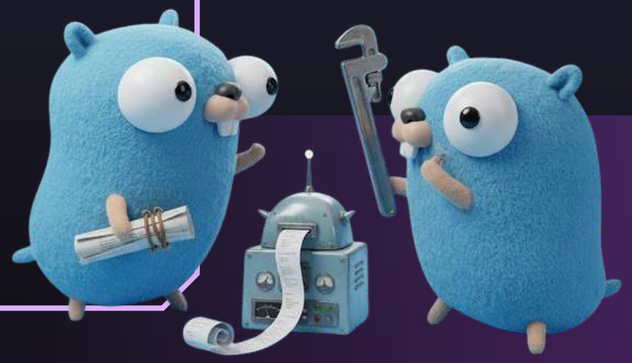
Channels: DeadLock

- If a Goroutine is sending data on a channel, then it is expected that some other Goroutine should be receiving the data

# CONCURRENCY

- Concurrency involves structuring a program so that two or more tasks may be in progress simultaneously



# PARALLELISM

- Parallelism allows for two or more tasks to be executed simultaneously
- Parallelism requires more than one processor or thread, concurrency does not

# CONCURRENCY VS PARALLELISM

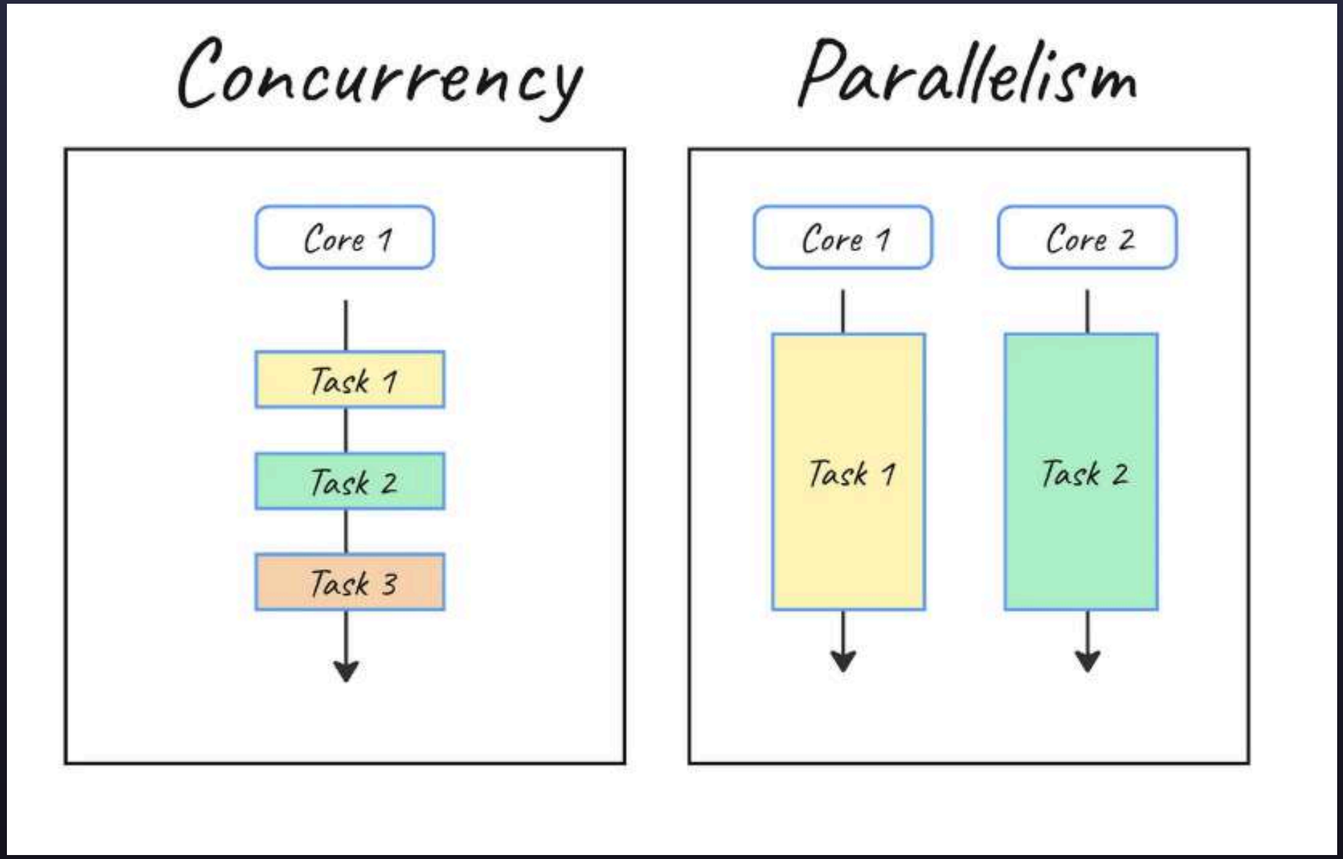
Concurrency : You are cooking

- Boiling rice ,Frying vegetables
- You switch between them.

Parallelism : You + your friend cooking

- You cook rice,Friend cooks vegetables
- Both happen at the same time

# CONCURRENCY VS PARALLELISM



WLUG
COMMUNITY | KNOWLEDGE | SHARE

# THANK YOU!